Python II 1

Εξαιρέσεις

Σφάλματα και Εξαιρέσεις

- Στον προγραμματισμό, συχνά συμβαίνουν λάθη, τα οποία όμως πρέπει να μάθουμε να εντοπίζουμε και να διορθώνουμε.
- Στην Python έχουμε δύο βασικές κατηγορίες λαθών, τα Συντακτικά Λάθη και τις Εξαιρέσεις.

Συντακτικά Λάθη:

- Πρόκειται για λάθη στη σύνταξη του προγράμματος, που η Python βλέπει καθώς ”διαβάζει” τον κώδικά μας πριν τον εκτελέσει.

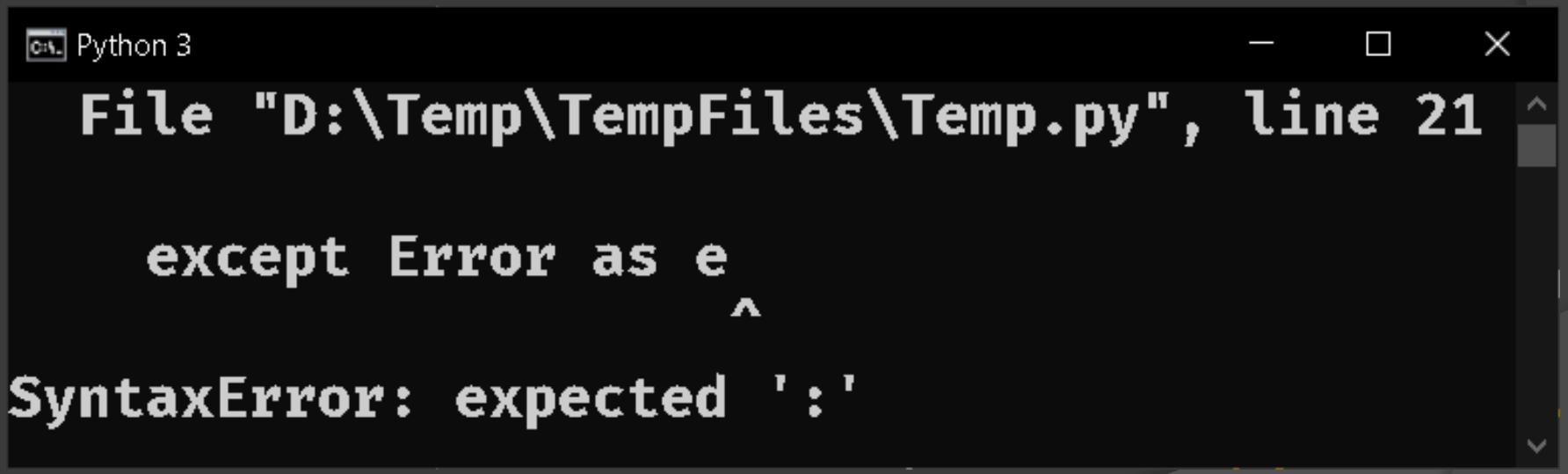
Σφάλματα και Εξαιρέσεις

Συχνές αιτίες λαθών περιλαμβάνουν:

1. Μη ισοσκελισμένες παρενθέσεις.
2. Κάποιο σημείο στίξης που λείπει (π.χ. άνω-κάτω τελεία ή κόμμα).
3. Λάθη στοίχισης (Εσοχές).
4. Κάποια εντολή που δε χρησιμοποιείται σωστά.

Σφάλματα και Εξαιρέσεις

- Από την Python πάντα λαμβάνουμε ένα κατάλληλο διαγνωστικό μήνυμα με το σημείο στο οποίο εντοπίζεται το λάθος.

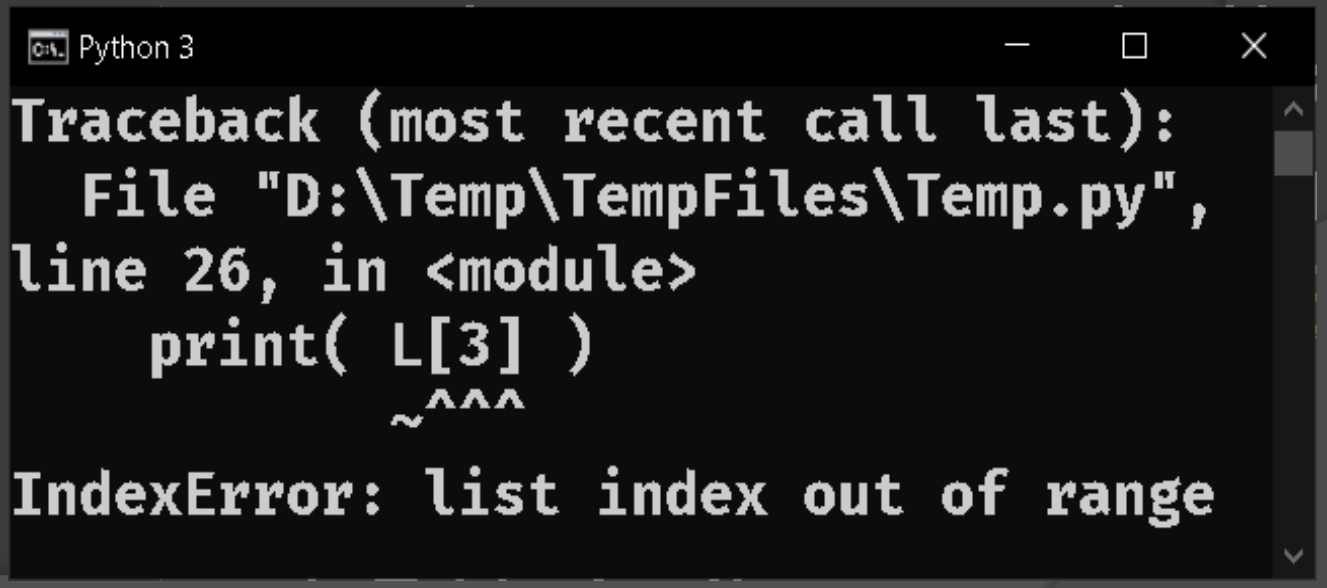


Εξαιρέσεις

- Ακόμα και όταν το πρόγραμμά μας είναι συντακτικά σωστό, ενδέχεται να εμφανιστούν λάθη κατά την εκτέλεση, τα οποία ονομάζονται Εξαιρέσεις (Exceptions).

Παράδειγμα:

```
L = [1, 2, 3]
print( L[3] )
```



Εξαιρέσεις

Συχνές αιτίες περιλαμβάνουν:

1. Χρήση μεταβλητών που δεν έχουν οριστεί.
2. Αριθμητικά λάθη (π.χ. διαίρεση με το μηδέν).
3. Χρήση δείκτη έξω από τα όρια λίστας.
4. Λάθη τύπων (π.χ. πρόσθεση μεταξύ ασύμβατων τύπων).
5. Λάθη τιμών (π.χ. μετατροπή String που περιέχει χαρακτήρες σε αριθμό).

Εξαιρέσεις

- Εκτός από τα παραπάνω, υπάρχουν πολλές ακόμα περιπτώσεις Εξαιρέσεων, όπως:
  1. Λάθη Εισόδου/Εξόδου.
  2. Λάθη σε Αρχεία,
  3. Κλπ.
- Μπορείτε να βρείτε την πλήρη λίστα των προκαθορισμένων εξαιρέσεων της Python στον παρακάτω σύνδεσμο:

https://docs.python.org/3/library/exceptions.html

Εξαιρέσεις

- Οι κυριότερες από τις Εξαιρέσεις τις λίστας είναι οι παρακάτω:

Α/Α	Εξαίρεση	Περιγραφή
1	**ValueError**	Η τιμή δεν είναι αποδεχτή.
2	**IndexError**	Ο δείκτης δεν υπάρχει στην δομή ακολουθίας.
3	**KeyError**	Πατήθηκε το λάθος πλήκτρο από το πληκτρολόγιο.
4	**OSError**	Μι συμβατό Λειτουργικό σύστημα για τον κώδικα.
5	**FileNotFoundError**	Το αρχείο δεν βρέθηκε στην προκαθορισμένη θέση.
6	**IOError**	Λάθος Εισόδου/Εξόδου δεδομένων (αρχείων).

Εξαιρέσεις

- Οι κυριότερες από τις Εξαιρέσεις τις λίστας είναι οι παρακάτω:

Α/Α	Εξαίρεση	Περιγραφή
7	**ZeroDivisionError**	Διαίρεση με το 0.
8	**TypeError**	Ο τύπος δεδομένων δεν είναι αποδεκτός.
9	**Exception**	Για λάθη που δεν καλύπτονται από κάποια Εξαίρεση (αγνώστου τύπου Εξαίρεση).

Χειρισμός Εξαιρέσεων

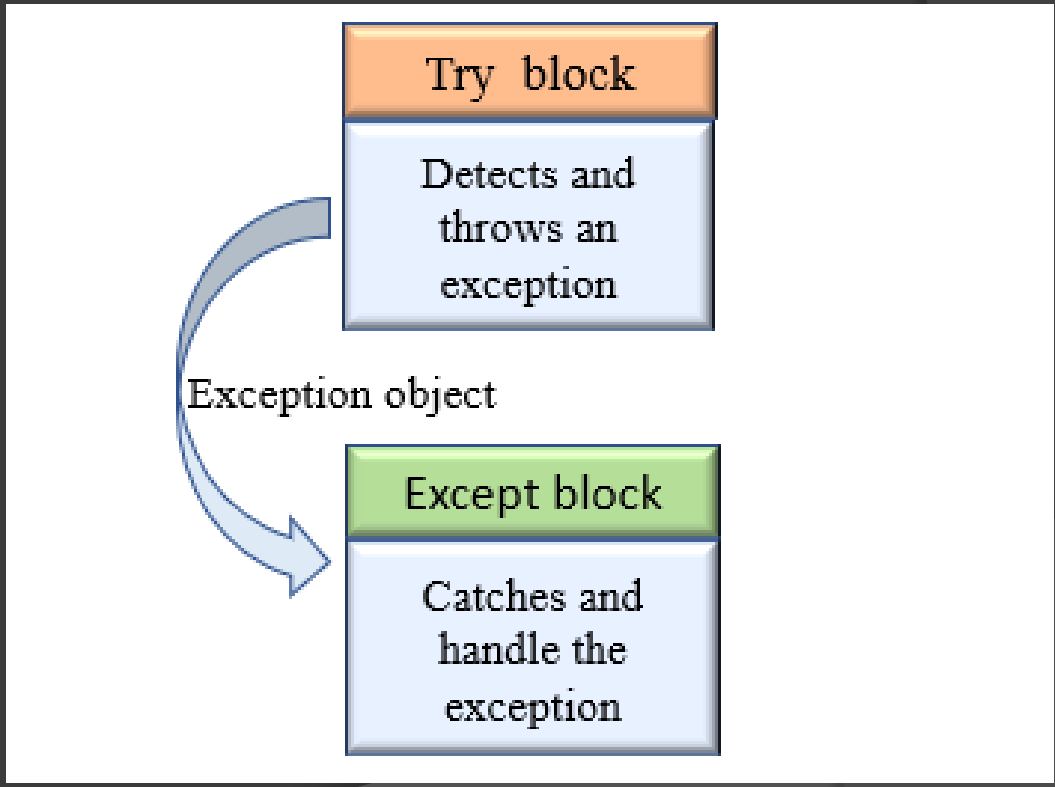
- Όπως είπαμε, όταν συμβαίνει μια Εξαίρεση το πρόγραμμα τερματίζει δείχνοντας το μήνυμα λάθους.
- Υπάρχουν όμως περιπτώσεις που μπορούμε να διαχειριστούμε κάποιες Εξαιρέσεις ώστε το πρόγραμμα να ”αναρρώσει” από το λάθος και να συνεχίσει την εκτέλεσή του.
- Αυτό γίνεται με τις ειδικές εντολές

  try και except.

Χειρισμός Εξαιρέσεων

Το γενικό συντακτικό είναι το ακόλουθο:

```
try:
    Εντολή try-1
    Εντολή try- 2
    ……
except Error_Type:
    Εντολή exept_1.1
    Εντολή exept_1.2
  ……
```



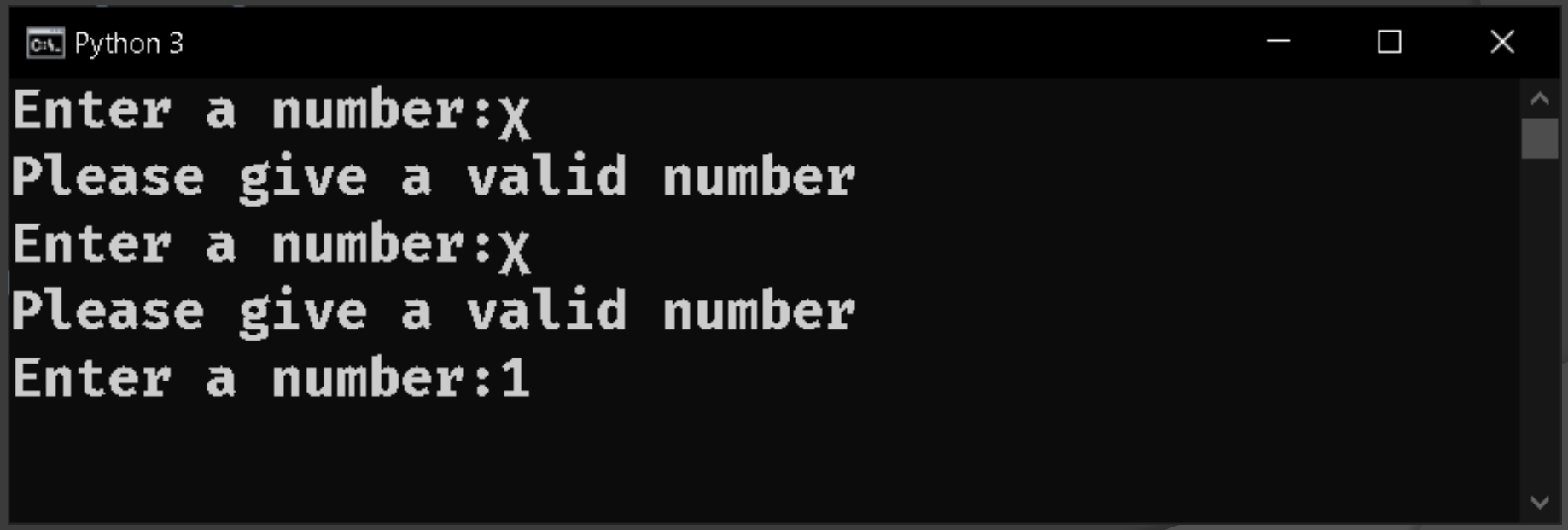
Χειρισμός Εξαιρέσεων

- Για παράδειγμα, το παρακάτω πρόγραμμα χειρίζεται περιπτώσεις Λάθους Εισόδου (ValueError) και ξαναζητάει είσοδο μέχρι να δοθεί ένας ακέραιος αριθμός.

```
while True:
    try:
        x = int(input('Enter a number: ’))
        break
    except ValueError:
        print("Please give a valid number")
```

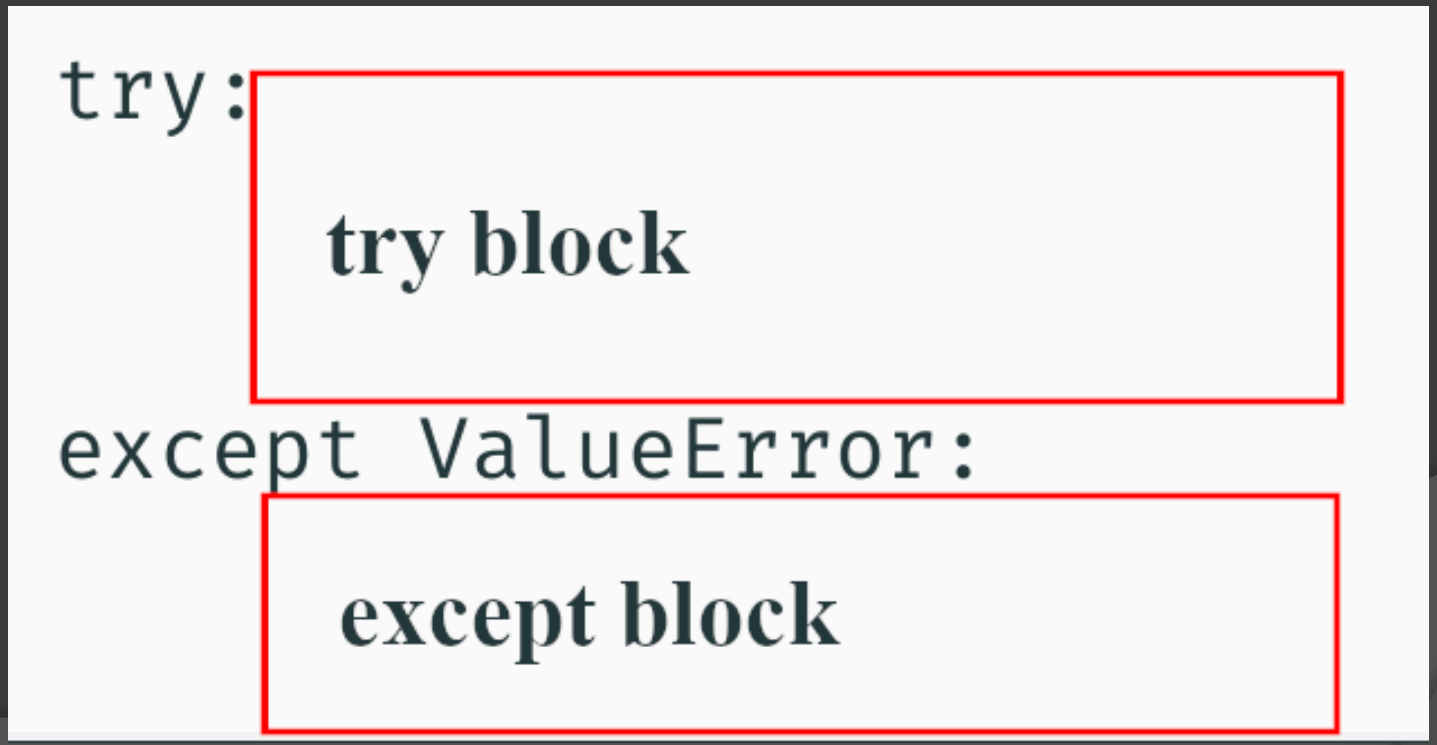
Χειρισμός Εξαιρέσεων

- Το αποτέλεσμα θα είναι το παρακάτω:



Χειρισμός Εξαιρέσεων

- Οι εντολές try και except ορίζουν τις αντίστοιχες περιοχές try (try block) και περιοχές except (except block).



Χειρισμός Εξαιρέσεων

- Για τη διαχείριση Εξαιρέσεων γράφουμε τον κώδικα ως εξής:
  1. Βάζουμε μέσα σε try block τον κώδικα προς Εκτέλεση.
  2. Ορίζουμε μέσα σε except block τις Εξαιρέσεις που μας ενδιαφέρουν και μπορεί να συμβούν μέσα στον κώδικα του try block, καθώς και τον κώδικα που θέλουμε να εκτελέσουμε σε περίπτωση που συμβεί η κάθε Εξαίρεση.

Χειρισμός Εξαιρέσεων

- Για τη διαχείριση Εξαιρέσεων γράφουμε τον κώδικα ως εξής:
  3. Εν γένει μπορούμε να έχουμε πολλαπλά except blocks, όμως μόνο ένα μπορεί να εκτελεστεί ανάλογα με τον τύπο της Εξαίρεσης.
  4. Προαιρετικά μπορούμε να έχουμε και ένα τρίτο block εντολών, το finally block το οποίο εκτελείται στο τέλος είτε προκλήθηκε Εξαίρεση είτε όχι.

Χειρισμός Εξαιρέσεων

Η εκτέλεση γίνεται ως εξής:

1. Εκτελούνται πρώτα οι εντολές του try block:
2. Αν δεν υπάρξει Εξαίρεση, ο κώδικας του try block ολοκληρώνεται με επιτυχία και παραλείπονται οι εντολές των όποιων except block.
3. Αν υπάρξει Εξαίρεση κατά τη διάρκεια της εκτέλεσης του try block (ή κώδικα που αυτό καλεί, άμεσα ή έμμεσα) παραλείπονται οι υπόλοιπες εντολές του try block.

Χειρισμός Εξαιρέσεων

Η εκτέλεση γίνεται ως εξής:

4. Αν η Εξαίρεση ταιριάζει με κάποια από τους δηλωμένους τύπους Εξαιρέσεων στα except blocks εκτελούνται οι εντολές του και στη συνέχεια εκτελείται η επόμενη εντολή έξω από τα try και except blocks.
5. Αν η Εξαίρεση δεν ταιριάζει με τους δηλωμένους τύπους Εξαιρέσεων, η εξαίρεση προωθείται σε πιθανά εξωτερικά (εμφωλευμένα) try blocks.

Χειρισμός Εξαιρέσεων

Η εκτέλεση γίνεται ως εξής:

6. Αν μια Εξαίρεση δεν ταιριάξει με κανένα except block, όσο έξω κι αν πάμε, τότε δε χειρίζεται από το πρόγραμμα και η εκτέλεση τερματίζει βγάζοντας μήνυμα λάθους όπως κι αν δεν είχαμε καθόλου try block.

Χειρισμός Εξαιρέσεων

Η εκτέλεση γίνεται ως εξής:

7. Αν υπάρχει finally block, ο κώδικάς του εκτελείται πάντα.
   a) Μετά το τέλος ενός επιτυχημένου try block
   b) Μετά το τέλος ενός ταιριασμένου except block.
   c) Πριν προωθηθεί μια μη ταιριασμένη Εξαίρεση σε εξωτερικό try block.

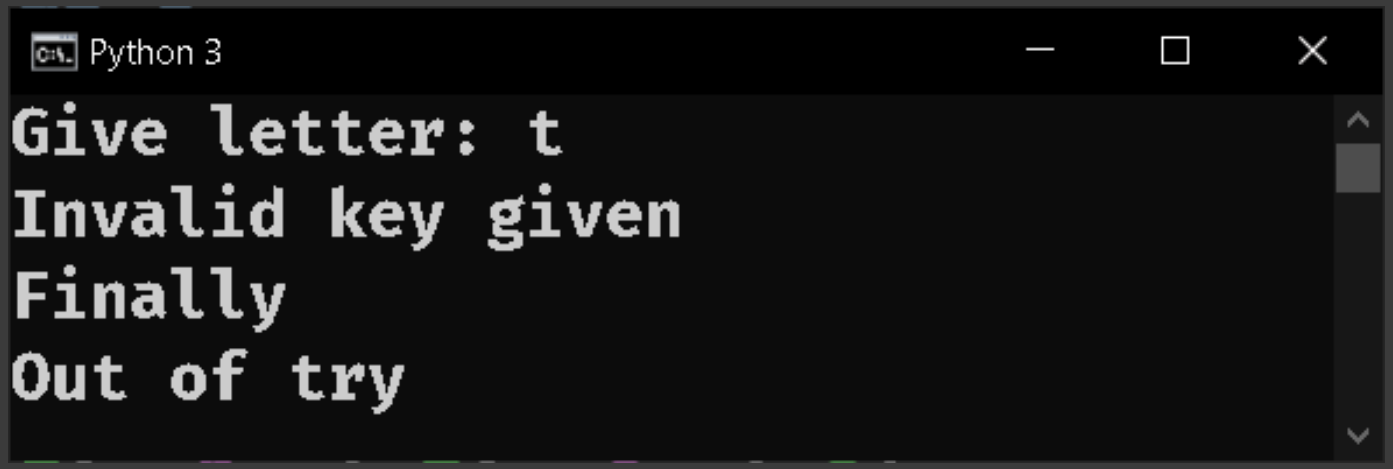
Χειρισμός Εξαιρέσεων

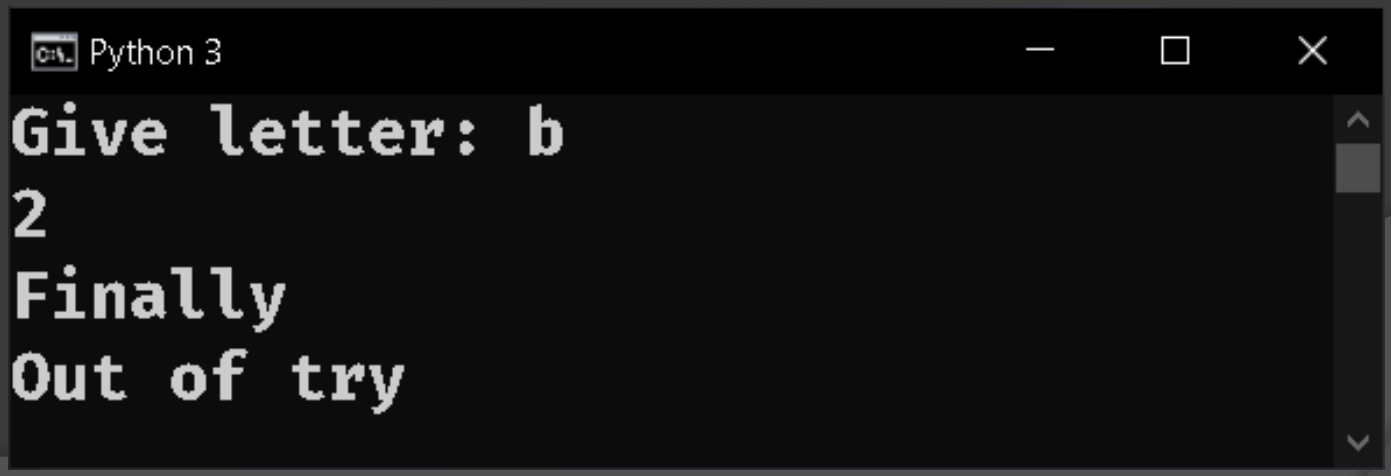
Παράδειγμα με try, except και finally:

```
try:
        D = {'a' : 1, 'b' : 2, 'c' : 3}
        s = input('Give letter: ‘)
        print(D[s])
except ValueError:
        print('Invalid key given')
finally:
        print('Finally/nOut of try')
```

Χειρισμός Εξαιρέσεων

- Το αποτέλεσμα θα είναι το παρακάτω:





Χειρισμός Εξαιρέσεων

Παράδειγμα με πολλαπλά except:

```
try:
    L = [1, 2, 3]
    x = int(input('Give position: '))
    print(L[x])
except ValueError:
    print('Please give a number for index’)
…
```

Χειρισμός Εξαιρέσεων

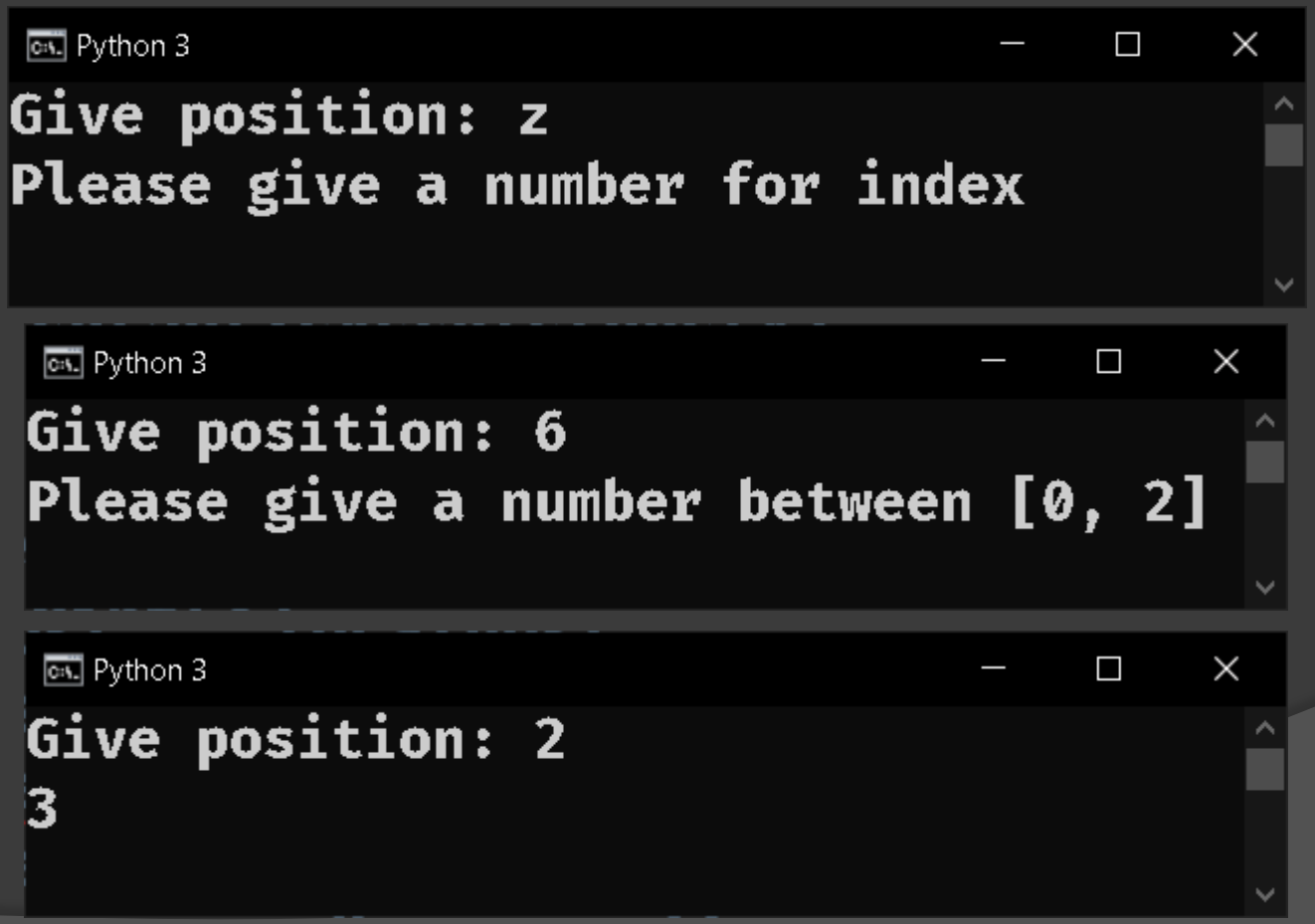
Παράδειγμα με πολλαπλά except:

…

```
except IndexError:
    print('Please give a number between [0, 2]')
except: # Για οποιοδήποτε άλλη εξαίρεση
    print('Unexpected error occurred’)
```

Χειρισμός Εξαιρέσεων

- Το αποτέλεσμα θα είναι το παρακάτω:



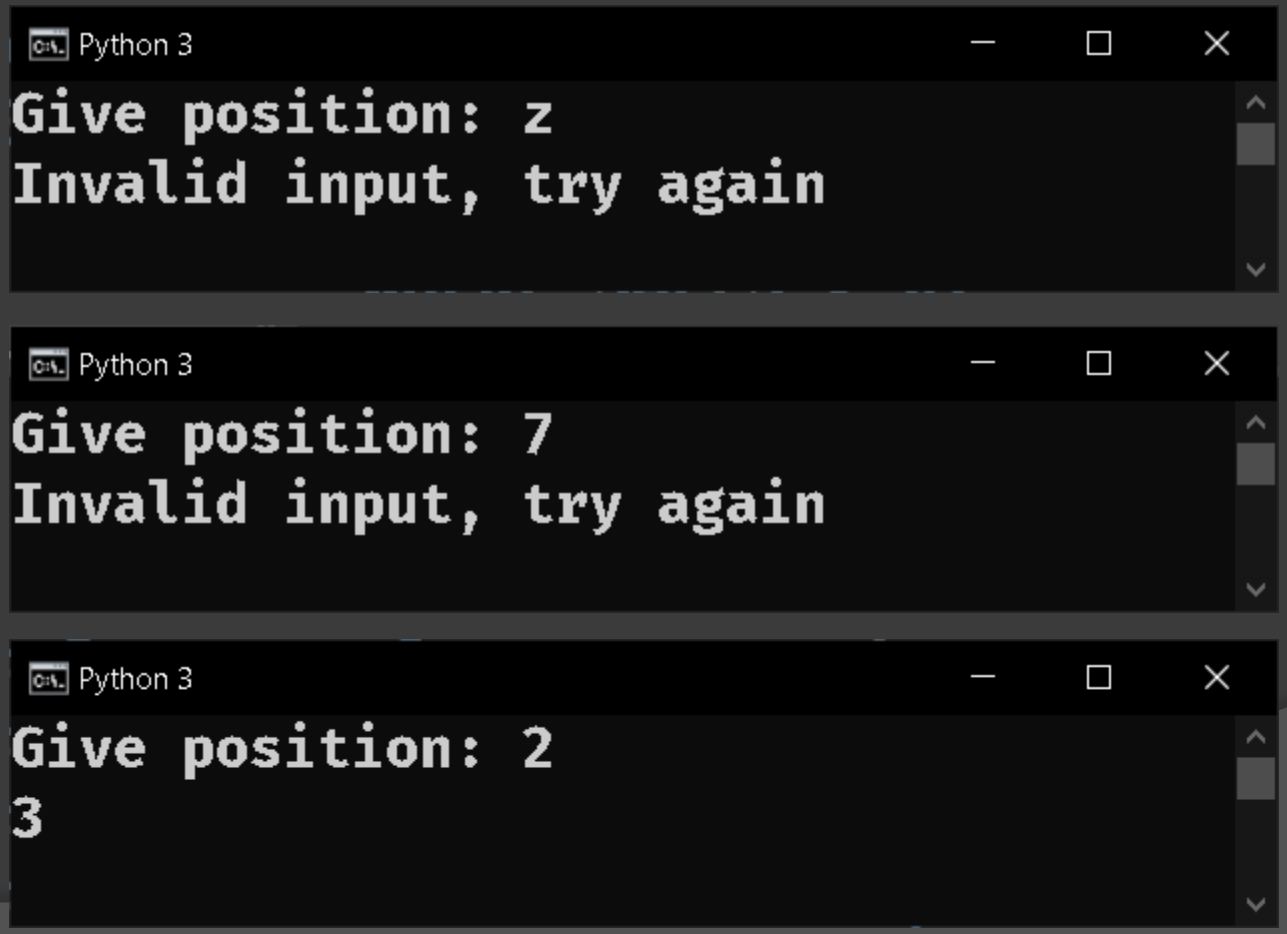
Χειρισμός Εξαιρέσεων

Παράδειγμα με πολλαπλές Εξαιρέσεις στο ίδιο except:

```
try:
    L = [1, 2, 3]
    x = int(input('Give position: '))
    print(L[x])
except (ValueError, IndexError):
    print('Invalid input, try again’)
```

Χειρισμός Εξαιρέσεων

- Το αποτέλεσμα θα είναι το παρακάτω:



Πρόκληση Εξαιρέσεων

- Μπορούμε κι εμείς να προκαλέσουμε Εξαιρέσεις στο πρόγραμμά μας χρησιμοποιώντας την εντολή raise.
- Στη raise δίνουμε το όνομα του τύπου της Εξαίρεσης που θέλουμε να προκαλέσουμε, ή ένα αντικείμενο της κλάσης αυτής.
- Προαιρετικά επίσης μπορούμε να προσθέσουμε και το μήνυμα λάθους που θα εμφανίζεται στο χρήστη.

Πρόκληση Εξαιρέσεων

- Το πλεονέκτημα εδώ είναι ότι μπορούμε να χρησιμοποιήσουμε τα Exceptions της Python, για λογικά λάθη που η ίδια η Python δεν μπορεί να εντοπίσει ή είναι για κριτήρια που ορίζουμε εμείς στον κώδικά μας.

Συντακτικό:

```
raise ValueError
raise NameError("Μήνυμα")
```

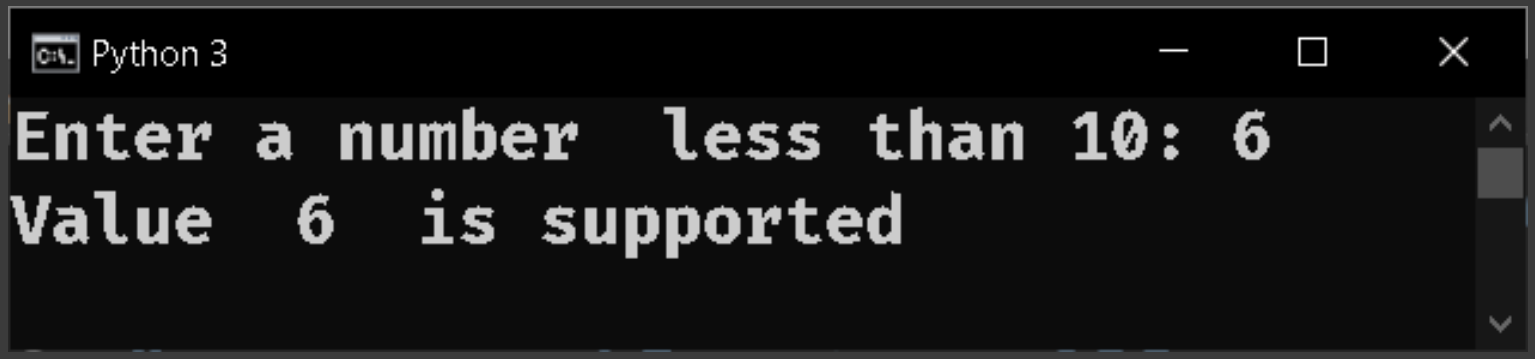
Πρόκληση Εξαιρέσεων

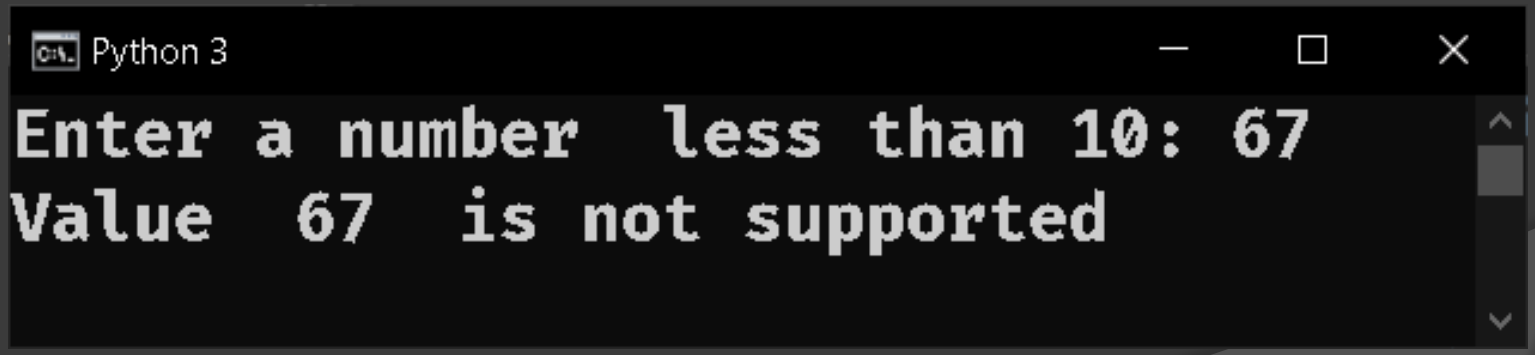
Παράδειγμα:

```
i=int(input("Enter a number  less than 10: "))
try:
        if i > 10:
            raise ValueError
        else:
            print("Value ", i, " is supported.")
except ValueError:
        print("Value ", i, " is not supported.")
```

Πρόκληση Εξαιρέσεων

- Το αποτέλεσμα θα είναι το παρακάτω:





Εκτύπωση Εξαίρεσης

- Η Python μας δίνει επίσης την δυνατότητα να εκτυπώνουμε ελεγχόμενα τα μηνύματα λάθους από τις Εξαιρέσεις δημιουργώντας ένα αντικείμενο της Κλάσης Εξαίρεσης.
- Αυτό επιτυγχάνεται χρησιμοποιώντας το τελεστή as και ένα όνομα για το αντικείμενο.
- Σε αυτήν την περίπτωση μπορούμε να εισάγουμε το αντικείμενο αυτό σε μια εντολή print() και να δούμε το μήνυμα λάθους στην οθόνη μας.

Εκτύπωση Εξαίρεσης

Συντακτικό:

```
try:
        # Κώδικας Python
except Error as error:
        print("Error: ", error)
```

- Το αντικείμενο error στον κώδικα μας περιέχει το Exception, την περιγραφή δηλαδή ενός πιθανού λογικού λάθους που μπορεί να συμβεί, αν αυτό συμβεί.

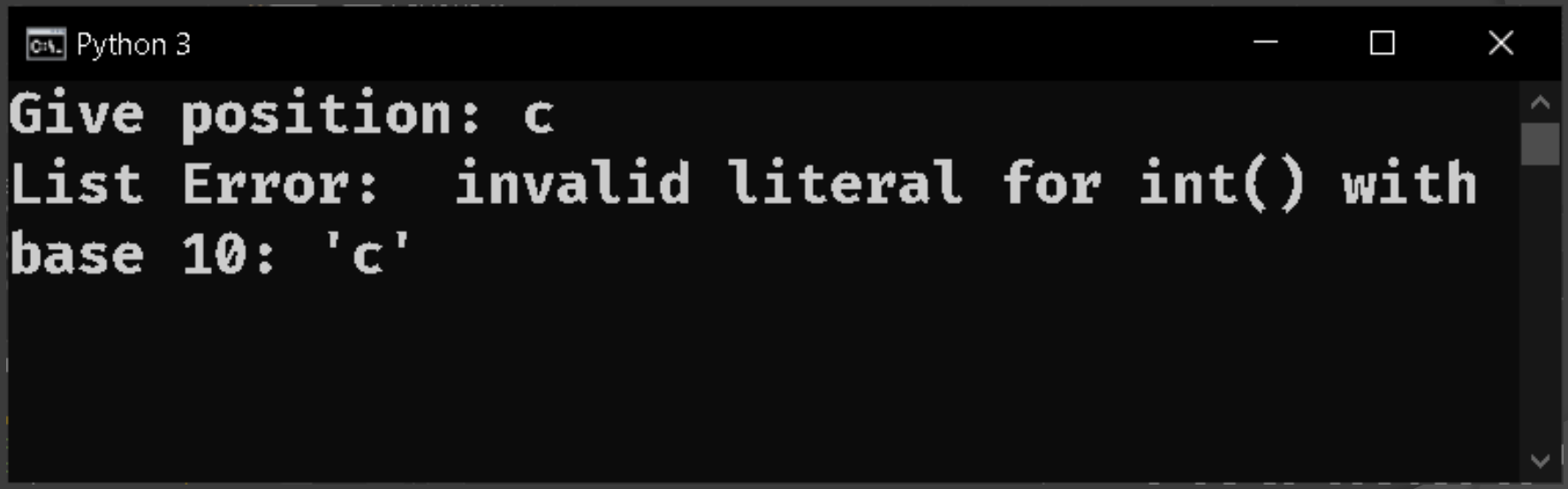
Πρόκληση Εξαιρέσεων

Παράδειγμα:

```
try:
    L = [1, 2, 3]
    x = int(input("Give position: "))
    print(L[x])
except ValueError as error:
    print("List Error: ", error)
```

Πρόκληση Εξαιρέσεων

- Το αποτέλεσμα θα είναι το παρακάτω:



PASS

- Η Python μας δίνει επίσης την δυνατότητα αν θέλουμε το πρόγραμμα μας να αγνοήσει τα Exceptions και να συνεχίσει την λειτουργία του δίχως να μας εμφανίσει κάποιο μήνυμα λάθους.
- Αν και αυτό δεν θεωρείται σωστή προγραμματιστική πρακτική σε κάποιες περιπτώσεις είναι χρήσιμη δυνατότητα.
- Το μόνο που έχουμε να κάνουμε είναι να προσθέσουμε την εντολή pass στο except block.

PASS

Παράδειγμα:

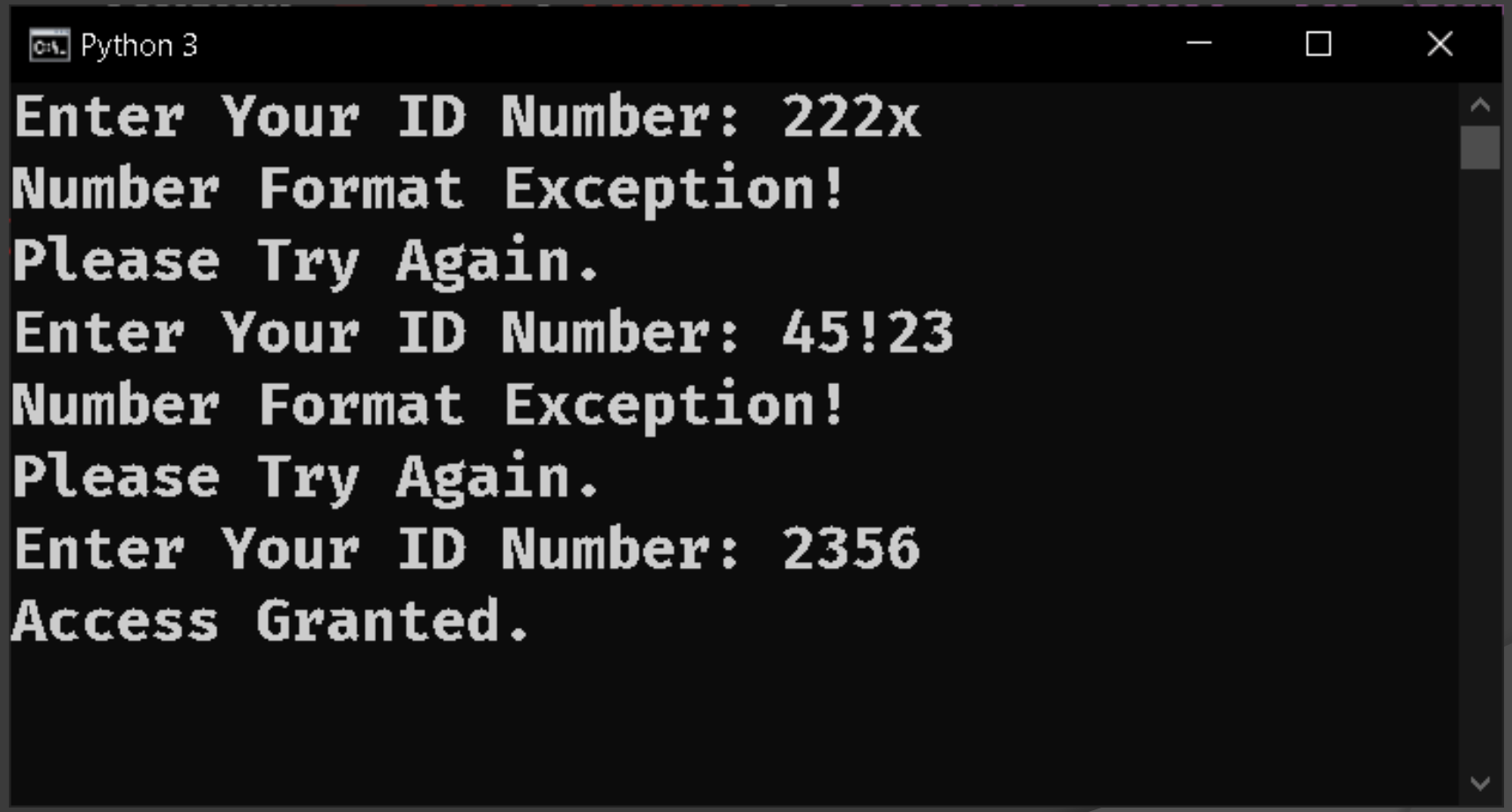
```
a = 0
div = 0
try:
        div = 23 / a
except:
        pass
print("Value of div is: ", div)
```



Άσκηση 1

- Γράψτε ένα πρόγραμμα Python πού θα ζητά από τον χρήστη να εισάγει τον αριθμό ID του για να του επιτραπεί η είσοδος στο πρόγραμμα.
- Αν ο χρήστης εισάγει κείμενο αντί για αριθμούς το πρόγραμμα θα βγάζει ένα μήνυμα λάθους με την χρήση του ValueError και θα ζητά από τον χρήστη να εισάγει ξανά το αριθμό ID του.

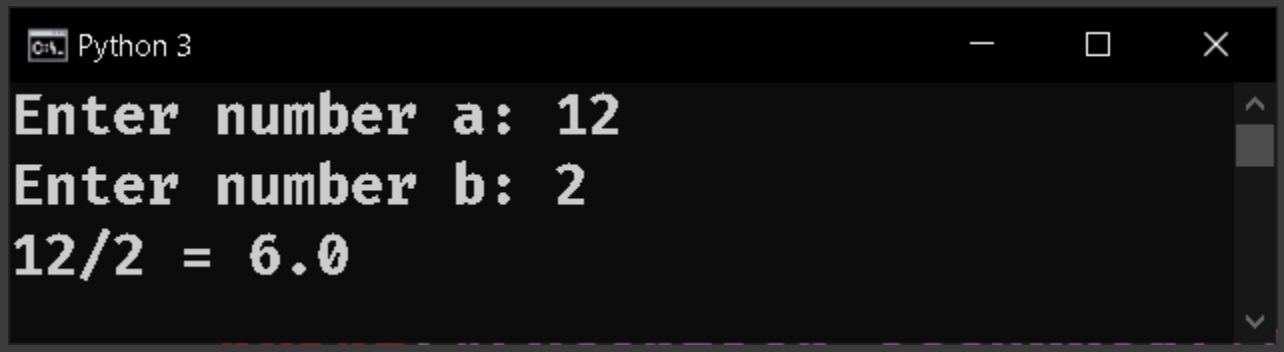
Άσκηση 1

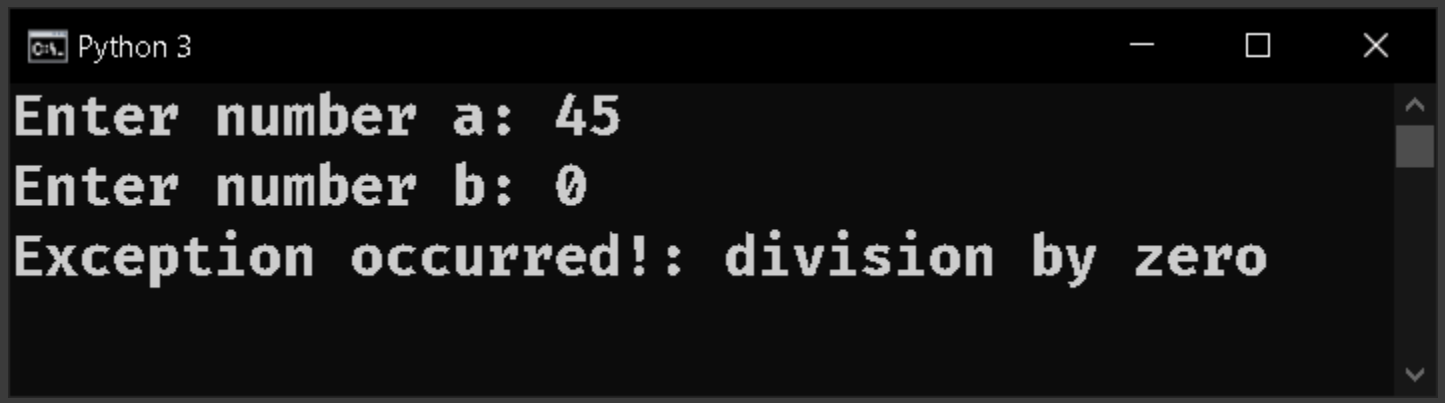

Python 3
Enter Your ID Number: 222x
Number Format Exception!
Please Try Again.
Enter Your ID Number: 45!23
Number Format Exception!
Please Try Again.
Enter Your ID Number: 2356
Access Granted.

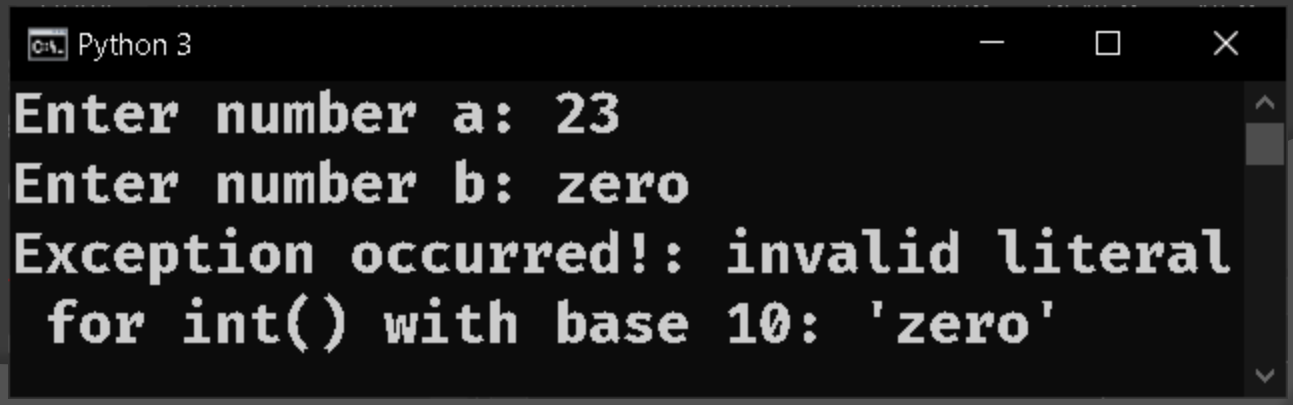
Άσκηση 2

- Γράψτε ένα πρόγραμμα Python πού θα δέχεται δύο ακέραιους αριθμούς και θα τους ελέγχει ως προς την ορθότητα των τιμών τους.
- Παράλληλά το πρόγραμμα θα πρέπει να μπορεί να διαχειριστεί Εξαιρέσεις του τύπου ZeroDivisionError και Exception για τα αντίστοιχα σφάλματα.

Άσκηση 2


Python 3
Enter number a: 12
Enter number b: 2
12/2 = 6.0

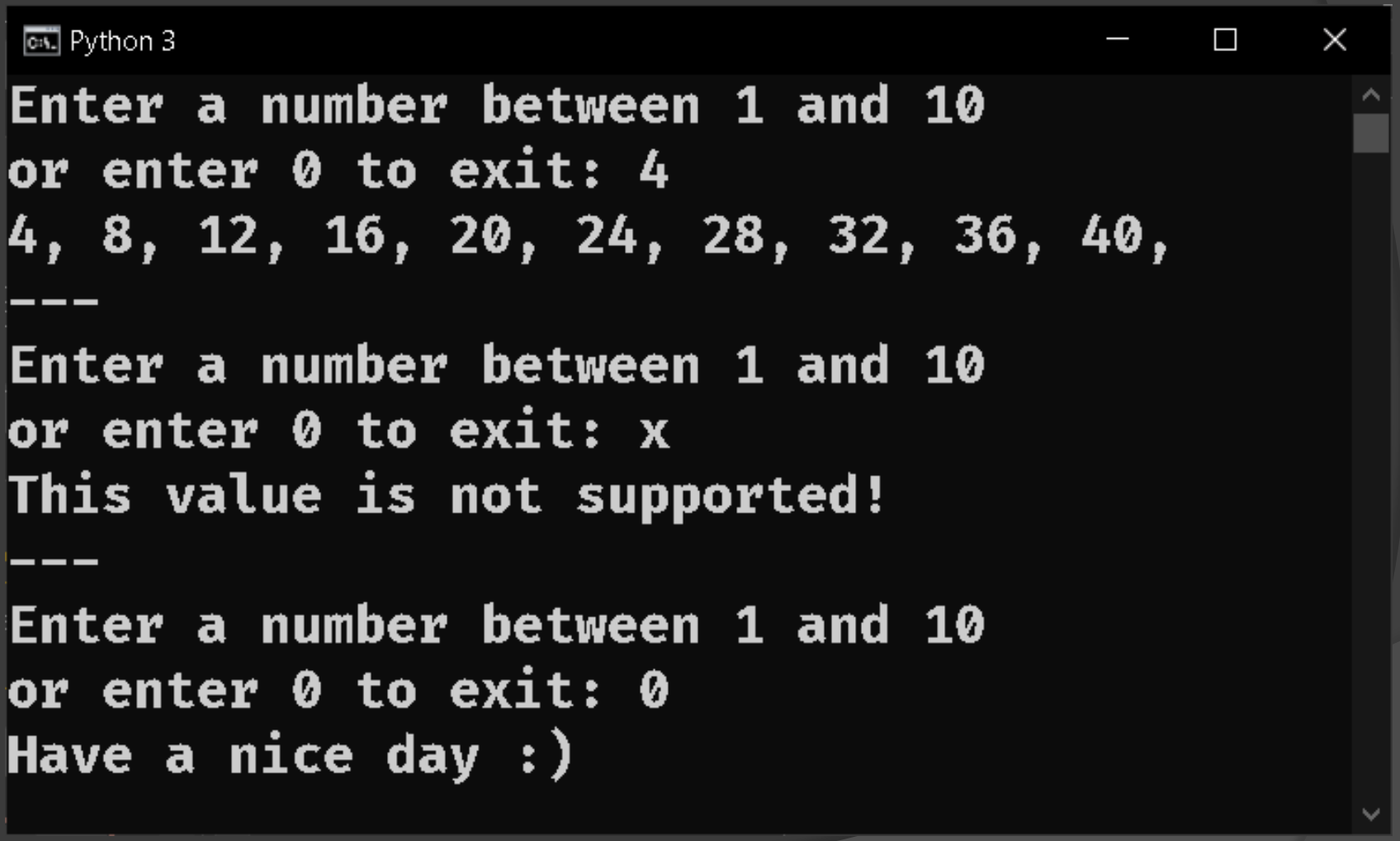

Python 3
Enter number a: 45
Enter number b: 0
Exception occurred!: division by zero


Python 3
Enter number a: 23
Enter number b: zero
Exception occurred!: invalid literal
for int() with base 10: 'zero'

Άσκηση 3

- Γράψτε ένα πρόγραμμα Python πού θα δέχεται από τον χρήστη έναν αριθμό από το 1 έως το 10 και το πρόγραμμα θα εμφανίζει τα 10 πρώτα πολλαπλάσια του αριθμού.
- Το πρόγραμμα θα σταματά μόνο όταν ο χρήστης εισάγει τον αριθμό 0 από το πληκτρολόγιο.
- Σε κάθε περίπτωση θα πρέπει το πρόγραμμα να ελέγχει την είσοδο του χρήστη αν είναι αποδεκτή η όχι.

Άσκηση 3


Python 3
Enter a number between 1 and 10
or enter 0 to exit: 4
4, 8, 12, 16, 20, 24, 28, 32, 36, 40,

Enter a number between 1 and 10
or enter 0 to exit: x
This value is not supported!

Enter a number between 1 and 10
or enter 0 to exit: 0
Have a nice day :)

Άσκηση 4

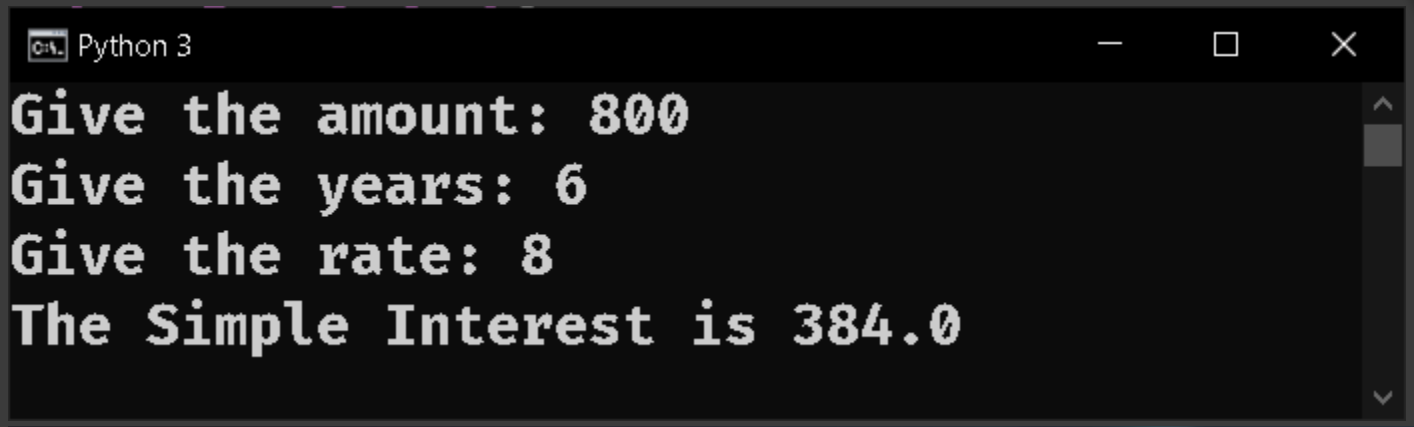
- Γράψτε ένα πρόγραμμα Python πού θα υπολογίζει τον τόκο ενός καταθέτη τράπεζας.
- Το πρόγραμμα θα δέχεται από τον χρήστη τρείς αριθμούς το ποσό κατάθεσης, τα χρόνια κατάθεσης και το επιτόκιο.
- Το πρόγραμμα θα υπολογίζει τα έχτρα λεφτά από τον τόκο χρησιμοποιώντας την παρακάτω έκφραση:

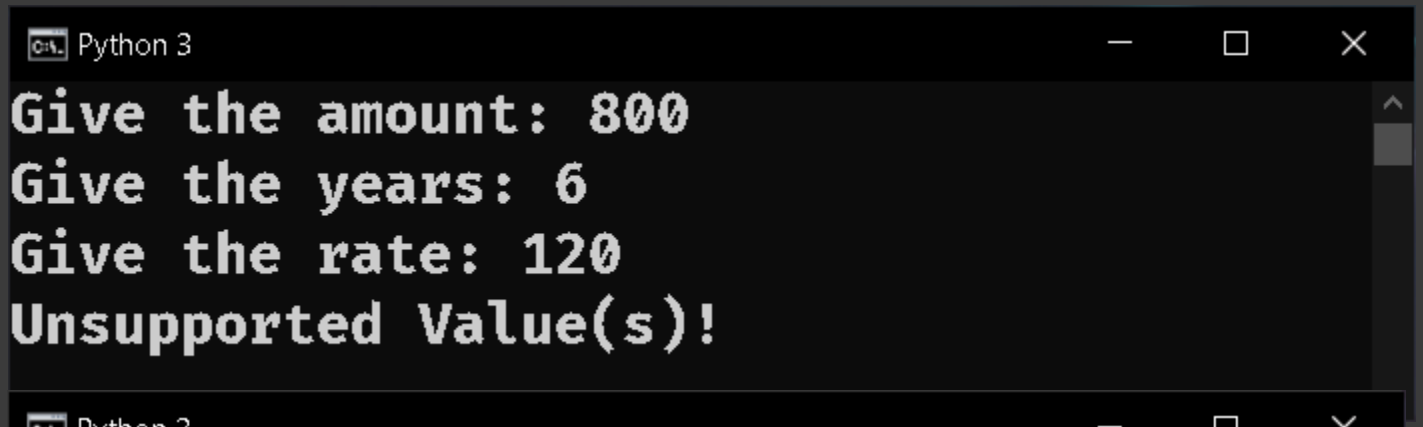
  (ποσό κατάθεσης * χρόνια * επιτόκιο) / 100
- Θα πρέπει:

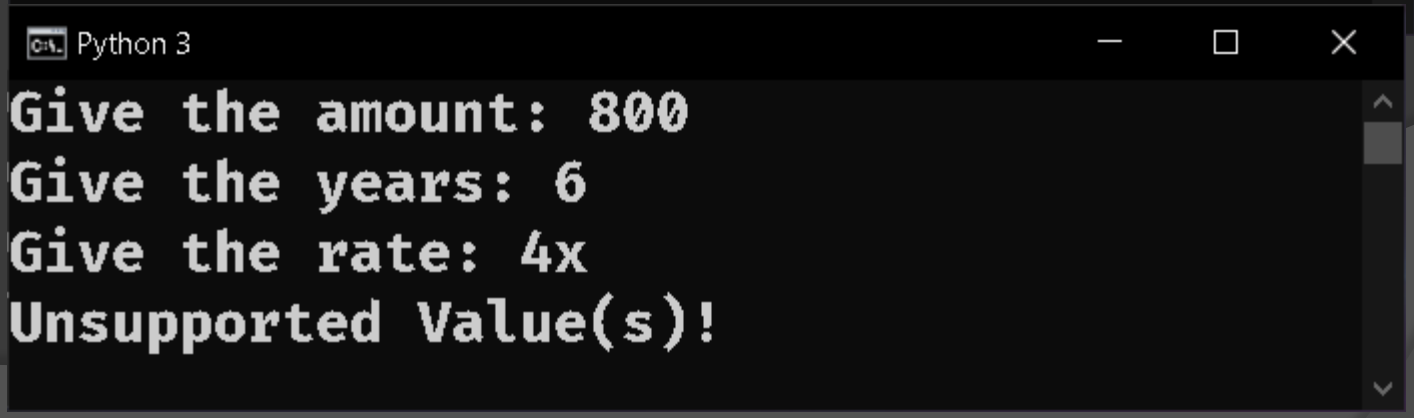
Άσκηση 4

1. Αν το επιτόκιο είναι μεγαλύτερο από 100 να εγείρουμε ένα ValueError Exception το οποίο θα προειδοποιεί τον χρήστη ότι το επιτόκιο που έκανε εισαγωγή δεν είναι αποδεκτό.
2. Να ελέγχει τις τιμές που κάνει εισαγωγή ο χρήστης αν είναι αριθμητικές ή όχι και να βγάζει το αντίστοιχο μήνυμα στην οθόνη.

Άσκηση 4


Python 3
Give the amount: 800
Give the years: 6
Give the rate: 8
The Simple Interest is 384.0


Python 3
Give the amount: 800
Give the years: 6
Give the rate: 120
Unsupported Value(s)!


Python 3
Give the amount: 800
Give the years: 6
Give the rate: 4x
Unsupported Value(s)!

Υποπρογράμματα

Υποπρογράμματα

- Ένα Υποπρόγραμμα είναι μια μονάδα / ενότητα προγράμματος που εκτελεί μια συγκεκριμένη εργασία.
- Αυτά τα Υποπρογράμματα συνδυάζονται για να σχηματίσουν μεγαλύτερα προγράμματα.
- Αυτό βασικά ονομάζετε 'Αρθρωτή Σχεδίαση'.

Υποπρογράμματα

- Ένα Υποπρόγραμμα μπορεί να καλείται από ένα άλλο πρόγραμμα, το οποίο ονομάζεται Πρόγραμμα Κλήσης.
- Η Python παρέχει δύο είδη Υποπρογραμμάτων:

1. Τις Συναρτήσεις (Functions)
2. Και τις Διαδικασίες (Procedures)

Υποπρογράμματα

1. Συναρτήσεις:
   Αυτά τα Υποπρογράμματα επιστρέφουν μια τιμή στο πρόγραμμα κλήσεων.

2. Διαδικασίες:
   Αυτά τα Υποπρογράμματα δεν επιστρέφουν καμία τιμή απευθείας στο πρόγραμμα κλήσεων.

Τοπικές και Καθολικές Μεταβλητές

Τοπικές Μεταβλητές

- Μια Τοπική Μεταβλητή ορίζεται μέσα σε μια Διαδικασία, Συνάρτηση ή σε μία Μέθοδο και είναι προσβάσιμη μόνο από εκεί.
- Λέγεται ότι έχει τοπική εμβέλια και δεν μπορεί να προσεγγιστεί από έξω.

```
def Dosomething
    Local_Var = 13
    return Local_Var
```

Τοπικές και Καθολικές Μεταβλητές

Καθολικές Μεταβλητές

- Μια Καθολική Μεταβλητή είναι μια μεταβλητή που δηλώνεται στην κύρια ενότητα ενός προγράμματος.
- Η Καθολική Μεταβλητή μπορεί να προσεγγιστεί από ένα πρόγραμμα ή άλλη μονάδα απευθείας και από παντού.

Τοπικές και Καθολικές Μεταβλητές

Καθολικές Μεταβλητές

Πχ.:

```
Global_Var = input("Enter your name: ")
print(Global_Var)
Dosomething(Global_Var)
```

Τοπικές και Καθολικές Μεταβλητές

Καθολικές Μεταβλητές

- Για να δημιουργήσουμε Καθολικές Μεταβλητές χρησιμοποιούμε την εντολή global.

Πχ.:

```
Global_Var = input("Please enter your name: ")
def Dosomething()
        global Global_Var
        Global_Var = "Tom"
print(Global_Var)
```

Συναρτήσεις

- Στην Python, μια συνάρτηση ορίζεται χρησιμοποιώντας τη εντολή def.
- Η γενική μορφή ενός ορισμού Συνάρτησης έχει ως εξής:

```
def Όνομα ( Παράμετροι ):
        <Κώδικας>
        return <Τιμή Επιστροφής>
```

Συναρτήσεις

- Η γενική μορφή ενός ορισμού Συνάρτησης έχει ως εξής:

```
// Συνάρτηση που ονομάζεται Main χωρίς παράμετροι
def Main():
// Συνάρτηση που ονομάζεται SayHello που δέχεται μία παράμετρο name τύπου String.
def SayHello( name ):
// Δέχεται δύο παραμέτρους που η μια είναι String, και h άλλη που είναι ακέραιος αριθμός.
def ShowMessage( message, times ):
```

Συναρτήσεις

Τιμή Επιστροφής:

- Όλες οι συναρτήσεις πρέπει να επιστρέφουν μια τιμή, οπότε σε όλες τις συναρτήσεις πρέπει να εκχωρηθεί μία Τιμή.
- Χρησιμοποιούμε τη εντολή return στο τέλος της συνάρτησης για να επιστρέψουμε την τιμή στο πρόγραμμα κλήσης.

Παράδειγμα

```
def Max(Num1, Num2):

    // Δήλωση Τοπικών Μεταβλητών
    Result = 0
    if (Num1 > Num2):
        Result = Num1
    else:
        Result = Num2
    return = Result
```

Συναρτήσεις

- Για να καλέσετε μια Συνάρτηση, απλά πρέπει να περάσετε τις απαιτούμενες Παραμέτρους μαζί με το Όνομα της Συνάρτησης και αφού η συνάρτηση επιστρέφει μια τιμή, χρειαζόμαστε και μια Μεταβλητή για να αποθηκεύσουμε τη Τιμή Επιστροφής.

Syndax:

Variable = Function_Name(Parameter(s))

Συναρτήσεις

- Ακολουθεί ένα απλό παράδειγμα χρήσης:

```
name = ReadString("Enter your name: ")

area = CircleArea(10.0)

maxBy2 = Highest(10, 3, 18) * 2

SayHello( ReadString("Enter your name") )
```

Συναρτήσεις

- Τα Απαιτούμενα Ορίσματα είναι τα Ορίσματα που εισέρχονται σε μια Συνάρτηση με τη σωστή σειρά.
- Στο παρακάτω παράδειγμα, ο αριθμός των Ορισμάτων στην κλήση Συνάρτησης δεν ταιριάζει ακριβώς με τον ορισμό της Συνάρτησης, με αποτέλεσμα να έχουμε προβλήματα στην εκτέλεση του κώδικα μας.

Συναρτήσεις

- Για να καλέσετε τη συνάρτηση printme, σίγουρα πρέπει να περάσετε ένα όρισμα, διαφορετικά η Python επιστρέφει ένα σφάλμα σύνταξης:

```
def printme( str ):
        print (str)

printme() # Error!
```

Συναρτήσεις

- Ένα Προεπιλεγμένο Όρισμα είναι μία παράμετρος που του ορίζουμε μια Προεπιλεγμένη Τιμή εάν ο χρήστης δεν δώσει μία τιμή για αυτό το όρισμα.
- Το ακόλουθο παράδειγμα δίνει μια ιδέα για τα Προεπιλεγμένα Ορίσματα, εκτυπώνει την προεπιλεγμένη ηλικία εάν ο χρήστης δεν περάσει συγκεκριμένη τιμή από το πληκτρολόγιο:

Συναρτήσεις

```
def printinfo( name, age = 35 ):
        print ("Name: ", name)
        print ("Age ", age)

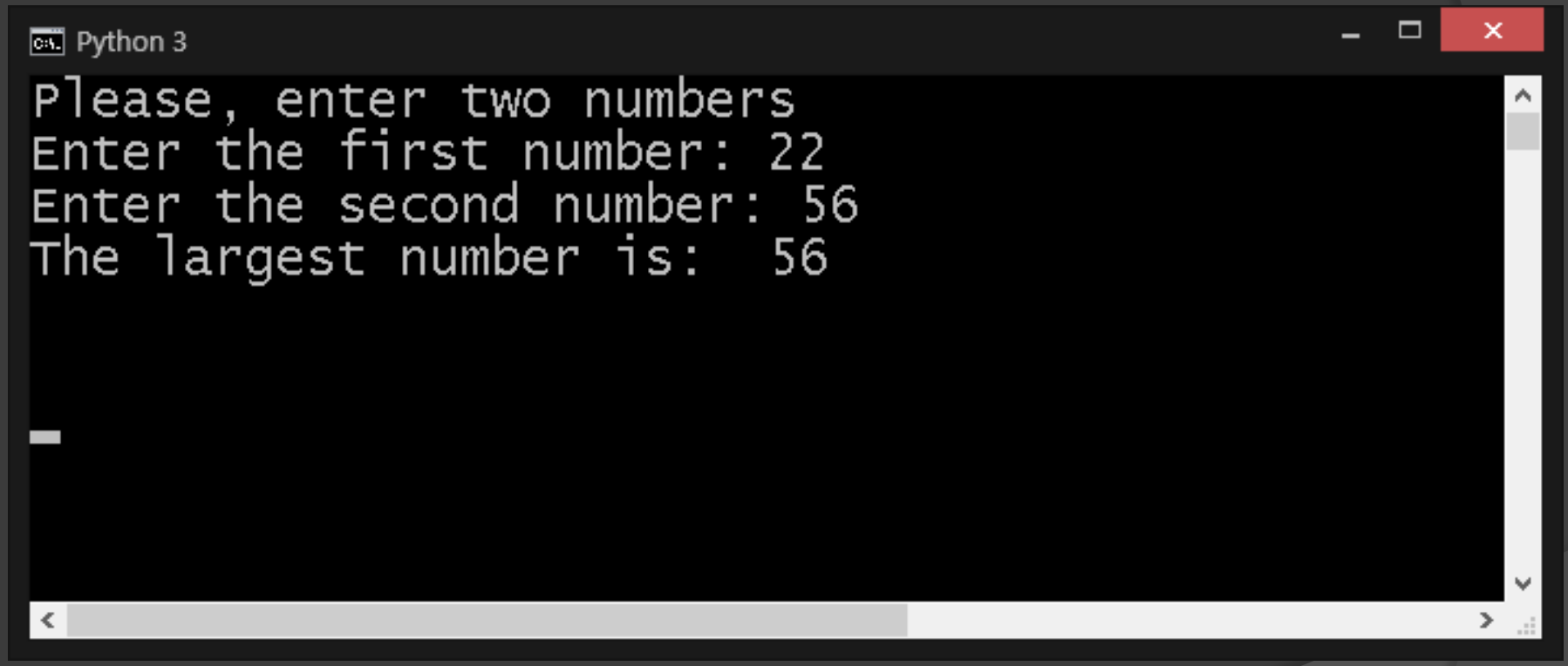
    # Τώρα μπορείτε να καλέσετε τη συνάρτηση
    #  printinfo

printinfo( age = 50, name = "Tom" )
printinfo( name = "Tom" )
```

Άσκηση 1

- Δημιουργήστε ένα πρόγραμμα που θα ζητά τον χρήστη να εισάγει δύο ακέραιους αριθμούς.
- Δημιουργήστε μια συνάρτηση με το όνομα Max που θα λαμβάνει ως παραμέτρους τους δύο αριθμούς και θα αποφασίζει ποιο από τα δύο είναι το μεγαλύτερο.
- Τέλος, η συνάρτηση θα επιστρέφει τον μεγαλύτερο αριθμό στο κύριο πρόγραμμα και θα εμφανίζεται ένα κατάλληλο μήνυμα.

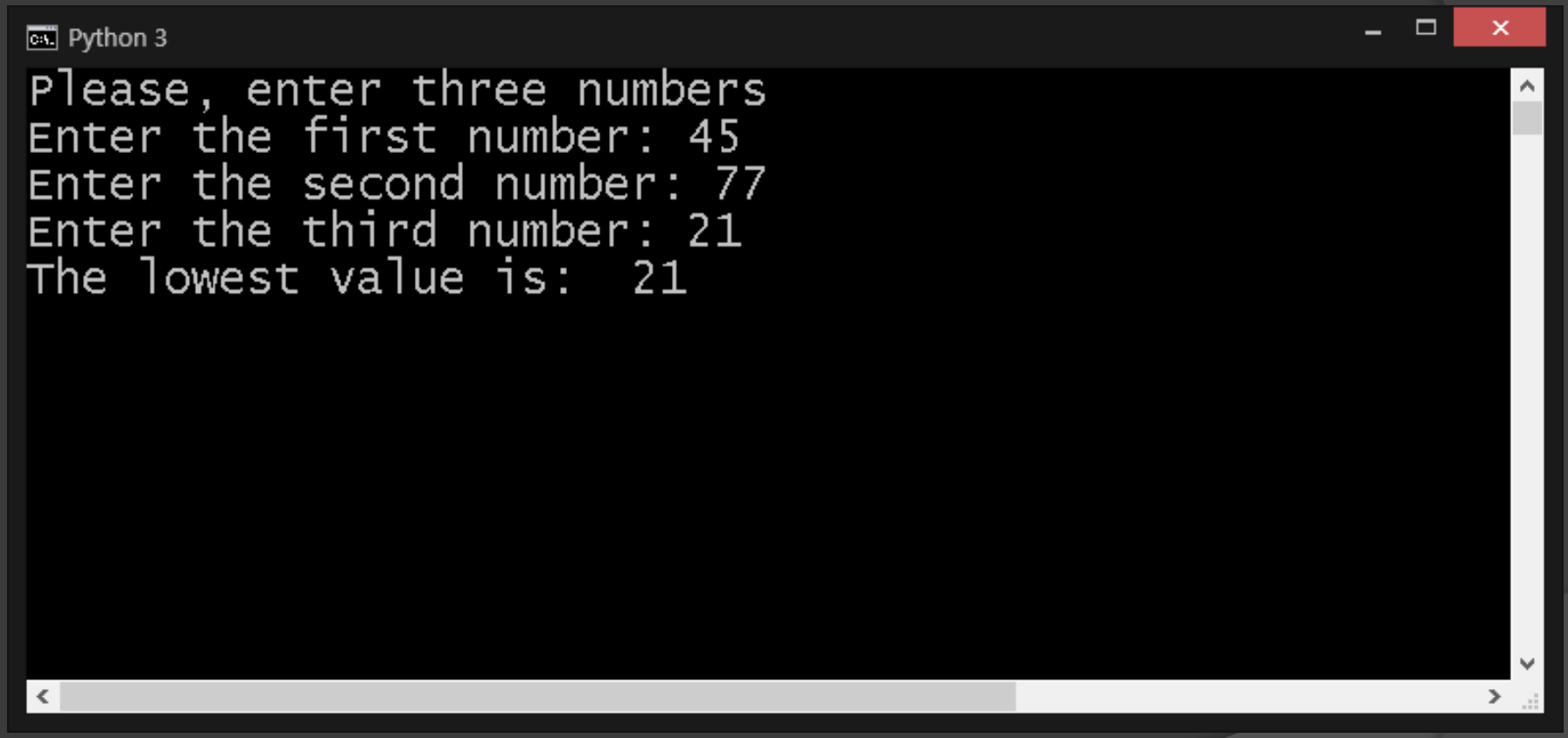
Άσκηση 1


Python 3
Please, enter two numbers
Enter the first number: 22
Enter the second number: 56
The largest number is: 56

Άσκηση 2

- Δημιουργήστε ένα απλό πρόγραμμα που θα ζητήσει από τον χρήστη να εισαγάγει τρεις ακέραιους αριθμούς.
- Δημιουργήστε μια διαδικασία με το όνομα FindMin που θα λαμβάνει ως παραμέτρους τους τρεις αριθμούς και θα αποφασίσει ποια από τις τρεις είναι μικρότερη.
- Τέλος, η διαδικασία θα αποθηκεύσει τον μικρότερο αριθμό σε μια μεταβλητή που ονομάζεται Min και θα εμφανιστεί ένα κατάλληλο μήνυμα στην οθόνη.

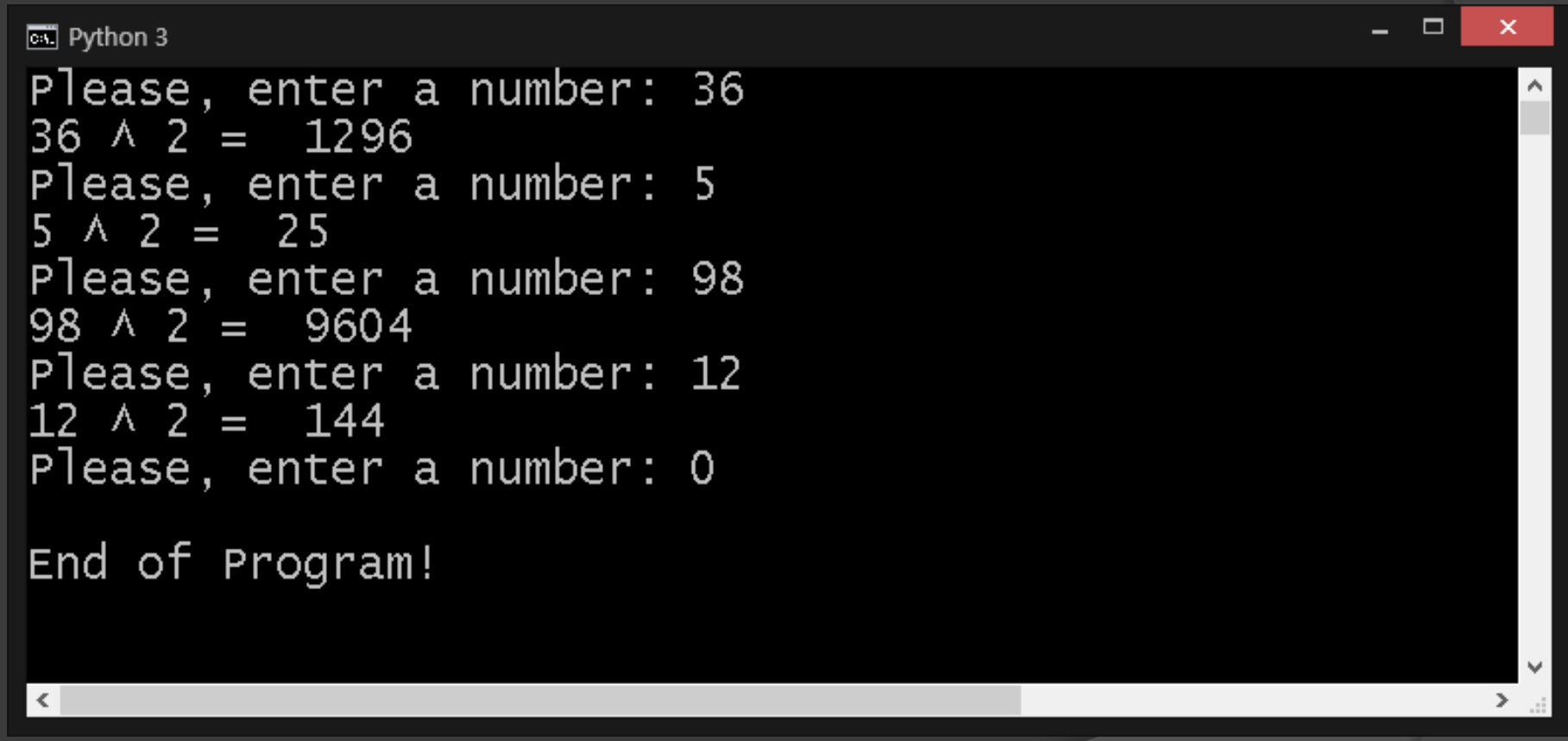
Άσκηση 2


Python 3
Please, enter three numbers
Enter the first number: 45
Enter the second number: 77
Enter the third number: 21
The lowest value is: 21

Άσκηση 3

- Δημιουργήστε ένα απλό πρόγραμμα που θα ζητά από τον χρήστη να εισαγάγει έναν ακέραιο αριθμό και εισάγετε μια διαδικασία με το όνομα Square που θα λαμβάνει ως παράμετρο τον αριθμό και θα υπολογίζει το τετράγωνο (X^2) του αριθμού και θα αποθηκεύει το τετράγωνο του αριθμού σε μια μεταβλητή με όνομα SQR και θα εμφανίζει ένα κατάλληλο μήνυμα στην οθόνη.
- Το πρόγραμμα θα λειτουργεί έως ότου ο χρήστης εισαγάγει τον αριθμό 0.

Άσκηση 3


Python 3
Please, enter a number: 36
36 ^ 2 = 1296
Please, enter a number: 5
5 ^ 2 = 25
Please, enter a number: 98
98 ^ 2 = 9604
Please, enter a number: 12
12 ^ 2 = 144
Please, enter a number: 0
End of Program!